



- ▶ As a consulting team, you need to provide a very high-level estimate to an organization for a particular scenario they are looking to solve
- ▶ You will use the AWS Pricing Calculator to put together your estimate
- ▶ Each team will be assigned a scenario, which describes the task, but details might be missing
- ▶ Your job:
 - ▶ Define all the hardware you think is needed to architect the solution
 - ▶ Provide an estimate of the costs
 - ▶ Each team will share their estimates and assumptions made

- ▶ We learned how to use the AWS pricing calculator to put together cost estimates (you will need this for your project)
- ▶ A huge portion of what companies are spending on cloud is wasted
 - ▶ It's estimated that over \$26B of cloud waste in 2021 (IaaS only)
 - ▶ It's critical that they are using the cloud efficiently to minimize “cloud waste”



- ▶ AWS provides a host of native cloud cost optimization tools you can use to cap your spending
- ▶ AWS Costs Explorer
 - ▶ An easy-to-use interface that lets you visualize, understand, and manage your AWS costs and usage over time
- ▶ Costs Anomaly Detection
 - ▶ Leverages advanced Machine Learning technologies to identify anomalous spend and root causes, so you can quickly take action
- ▶ AWS Budgets
 - ▶ Helps you stay within your allocated cloud spending range by setting spending or usage caps for your projects



Auto-Scaling

Engineering Cloud Software Systems

Department of Software Engineering
Rochester Institute of Technology



Netflix



- ▶ How much do you think Netflix spends per month on AWS?
 - ▶ It's estimated about ~\$28 million/month
 - ▶ Uses more than 100,000 server instances
- ▶ It's not clear what type of server are used, but consider a few pricing examples:

Type	Price per hour	Price x 10,000	Price after 24 hours
A1.medium	\$0.0255	\$255	\$6,120
M4.16xlarge	\$3.20	\$32,000	\$768,000
G3s.xlarge	\$0.75	\$7,500	\$180,000
P3.16xlarge	\$24.48	\$244,800	\$5,875,200

- ▶ To manage costs effectively, it's essential to pay only for what you need and nothing more

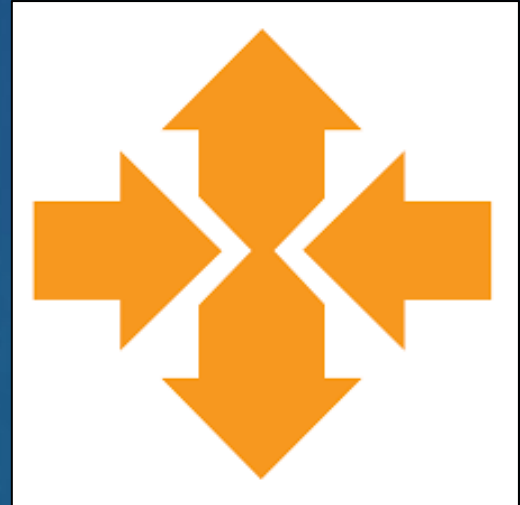
Elasticity

- ▶ Elasticity is one of the key principles of cloud computing
- ▶ It defines the ability to scale infrastructure up or down based on demand
- ▶ Defining the rules to programmatically scale up or down is referred to as Auto-Scaling
- ▶ It's a key advantage provided by cloud providers
 - ▶ Previously organizations were limited by the capacity in their data centers
- ▶ All major cloud providers (AWS, Google and Microsoft) offer Auto-Scaling capabilities



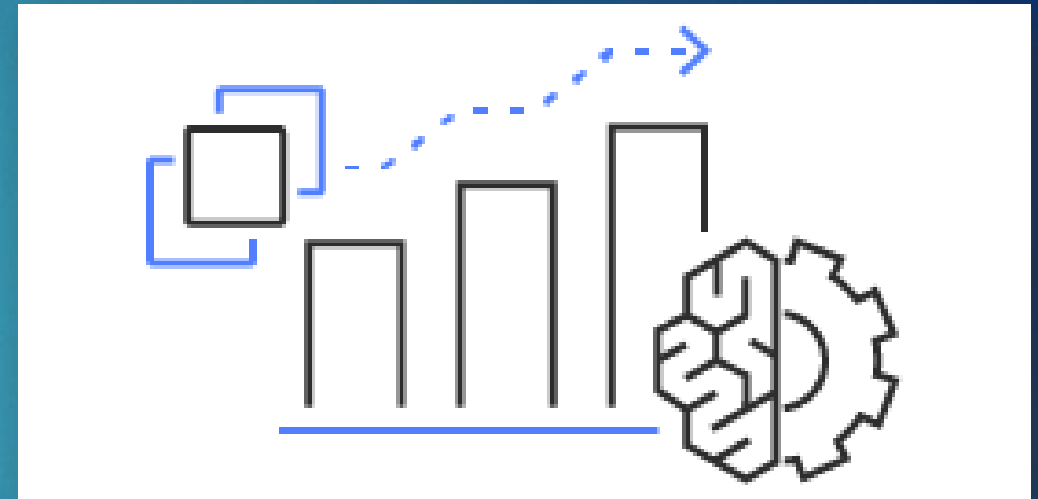
Auto-Scaling Advantages

- ▶ Fault tolerance
 - ▶ Auto-Scaling can detect when an instance is unhealthy, terminate it, and launch an instance to replace it
- ▶ Improved Availability
 - ▶ Auto-Scaling helps ensure that your application always has the right amount of capacity to handle the current traffic demand
- ▶ Cost management
 - ▶ When server loads are low, Auto-scaling allows companies managing their own infrastructure to automatically turn them off
 - ▶ This means paying for total usage instead of maximum capacity



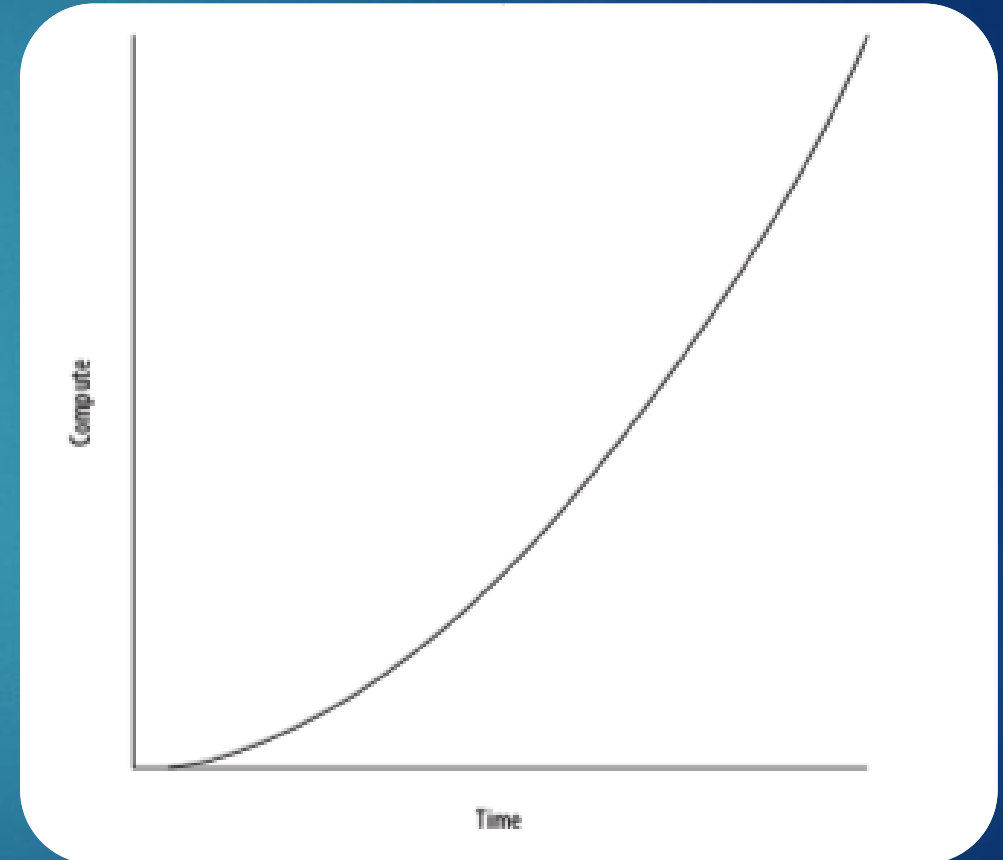
Auto-Scaling Usage Patterns

- ▶ Organizations will have different capacity needs based on usage patterns
- ▶ Examples include:
 1. Start Small, Grow Fast
 2. Predictable Burst
 3. Unpredictable Burst
 4. Periodic Processing
- ▶ Each requires a different type of plan to efficiently address the need



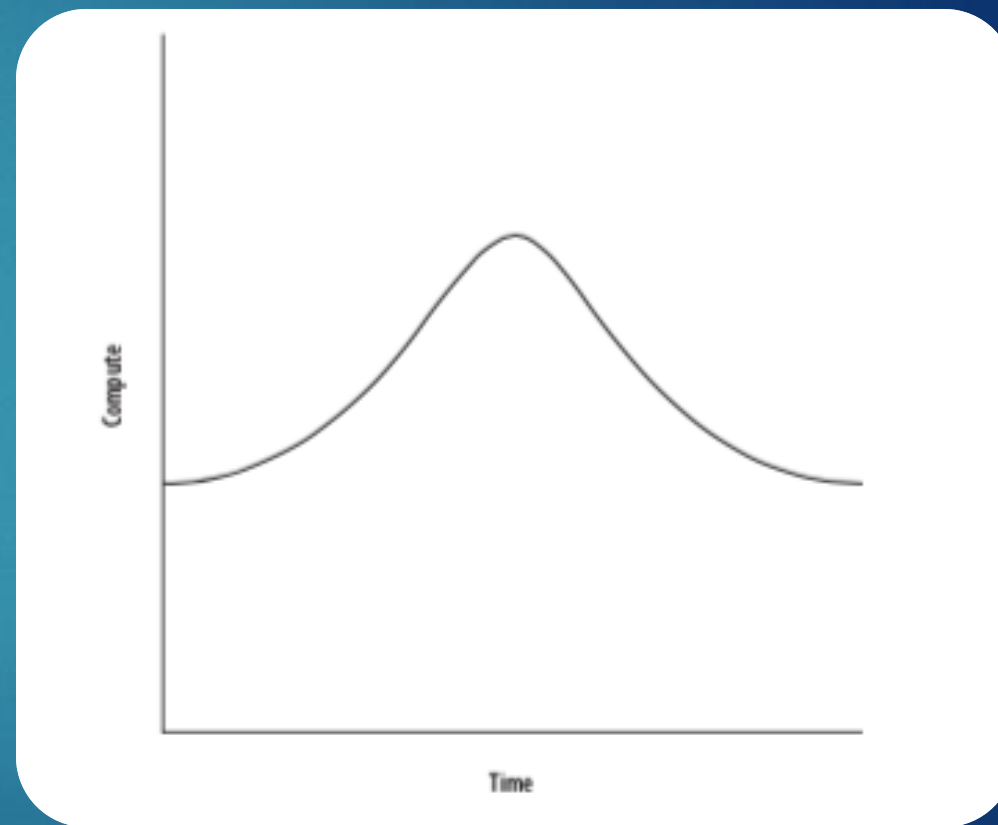
Start Small, Grow Fast

- ▶ Common scenario for startup companies that don't initially require a lot of capacity
- ▶ Cost is minimized until the demand grows
- ▶ As usage increases, the infrastructure must respond and scale up quickly to minimize any disruptions in service



Predictable Burst

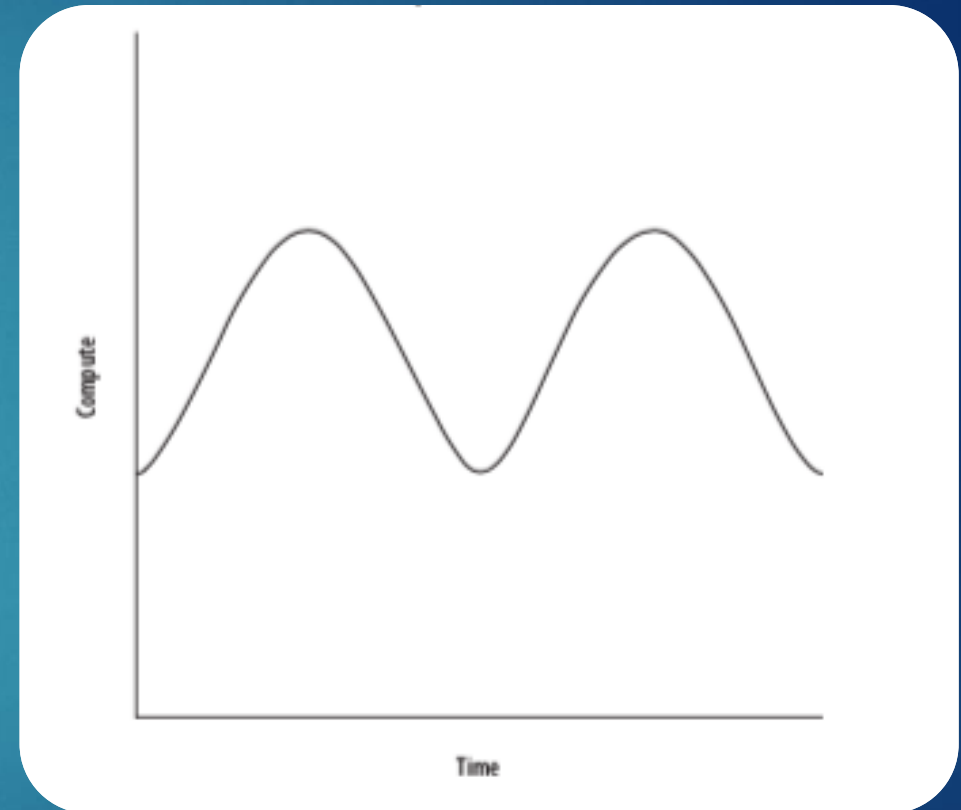
- ▶ Usually triggered by an event that may be anticipated
- ▶ For example, a new release of a product during a major event (e.g. Super bowl) may cause people to flood a web site
 - ▶ Shopping events (e.g. Holiday, Prime Day) is another common example
- ▶ Over time, the interest may wane and return to normal levels



Unpredictable Burst

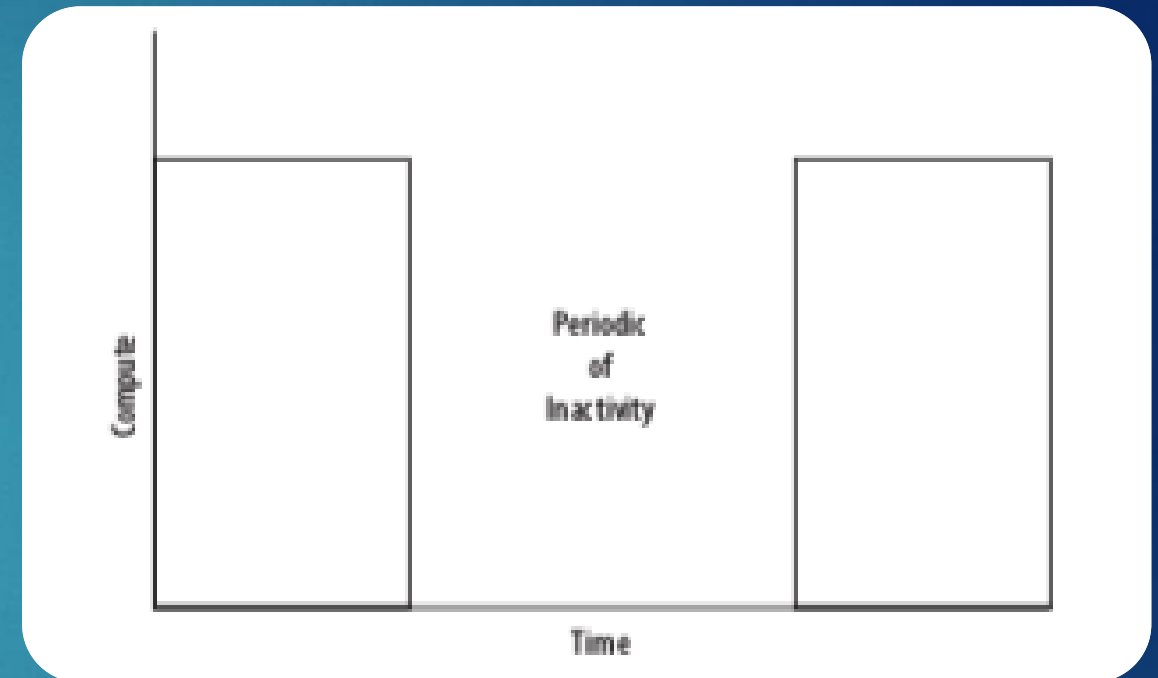
- ▶ Similar to a predictable burst, but not tied to an event that can be predicted
- ▶ An example could be a major news event such as a terrorist attack:

On September 11, 2001 websites such as CNN and MSNBC had removed their video, audio and photo feeds to ensure their pages could be loaded due to the overwhelming usage



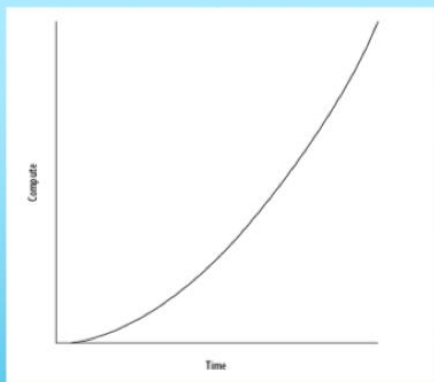
Periodic Processing

- ▶ Tied to an application that may be used heavily during a specific time of year and then forgotten completely
- ▶ For example, a tax application or a system processing annual reviews will spike in usage and completely drop afterwards

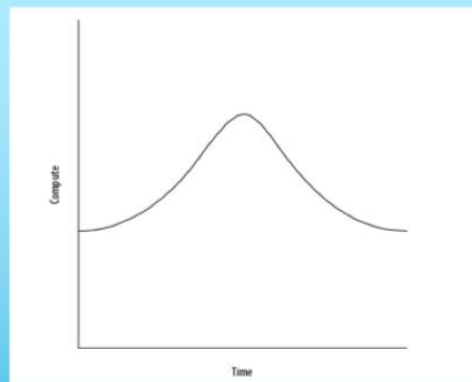


Sizing for Each Scenario

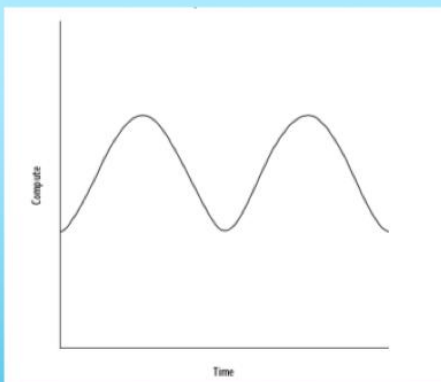
- ▶ Sizing for the anticipated peak and not adjusting can be expensive and is a common example of “cloud waste” (over-provisioning)
- ▶ Manually monitoring and adjusting the infrastructure up or down requires close attention to usage patterns, which would can be prone to errors



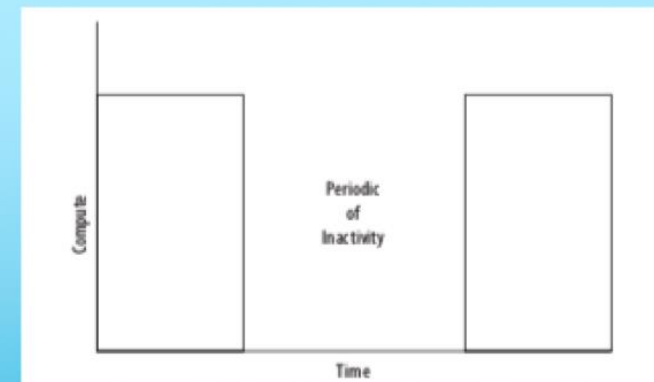
Start Small, Grow Fast



Predictable Burst



Unpredictable Burst



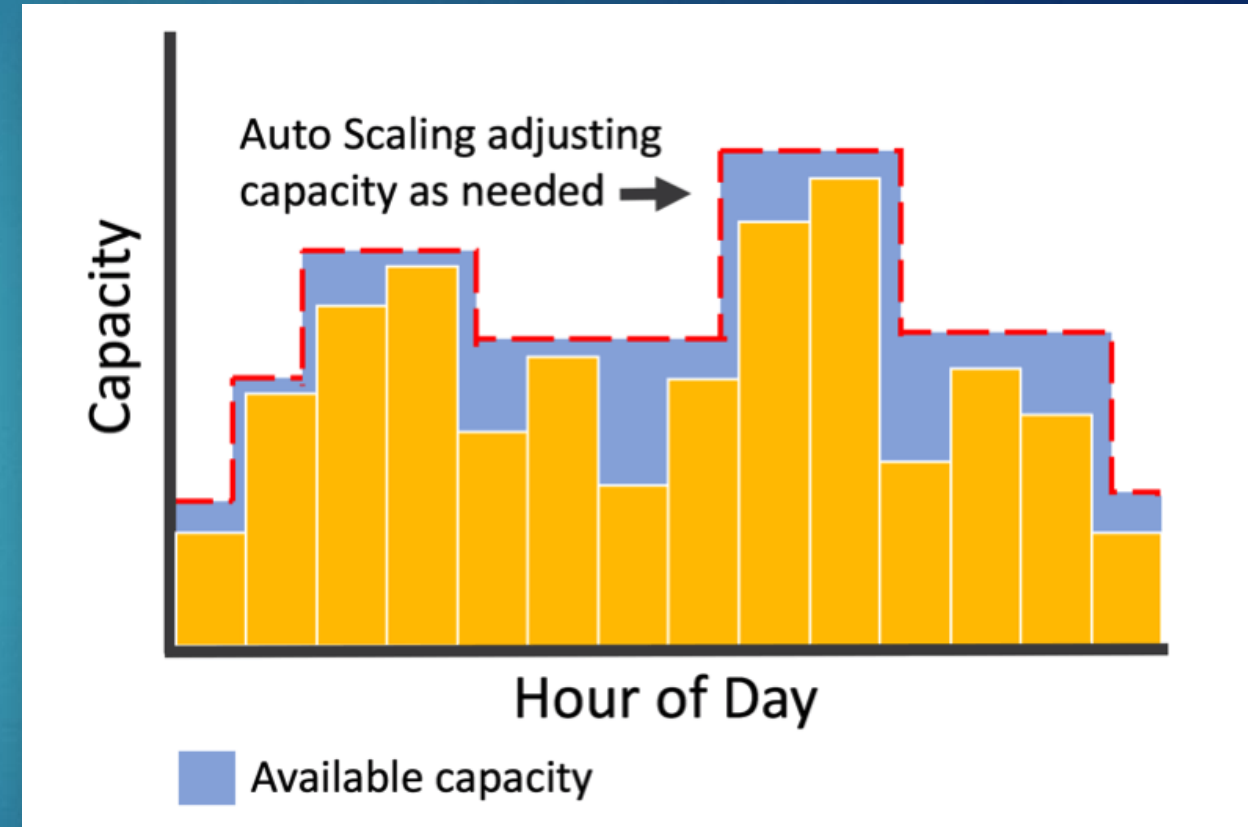
Periodic Processing

- ▶ This where Auto-Scaling can help

Auto-Scaling Overview

14

- ▶ Auto-scaling enables your resource group to dynamically increase or decrease capacity in response to real-time demand changes
- ▶ Instead of managing usage patterns manually, you simply set minimum and maximum threshold limits.
- ▶ The system automatically adjusts resources within those limits to maintain performance and efficiency



Auto-Scaling in AWS - 3 Main Components

15

1. Launch template

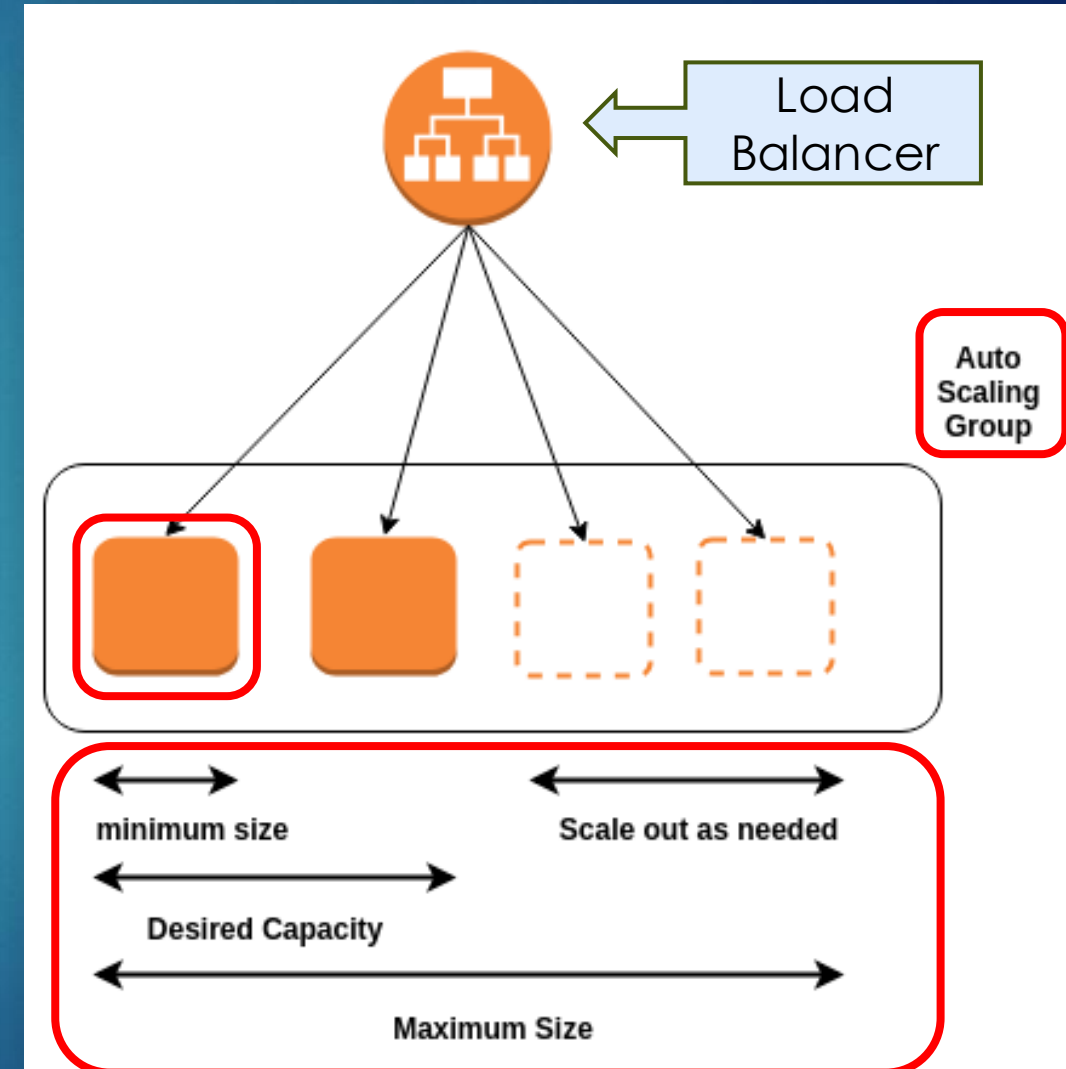
- ▶ Contains the AMI, instance type, etc. that Auto-Scaling groups use to launch the EC2 instances

2. Auto-Scaling group

- ▶ A collection of EC2 instances that are treated as a logical grouping for the purposes of automatic scaling and management

3. Auto-Scale plan

- ▶ Defines “when and how” to scale the Auto-Scaling group. Includes:
 - ▶ Health check time (how quickly to adjust)
 - ▶ Desired #, Min/Max #



Auto-Scaling Plan

- ▶ Auto-scaling strategies:
 - ▶ Always maintain current instance level
 - ▶ Configure and maintain a specified number of running instances at all the time. If a server goes down, a new one is launched
 - ▶ Scale based on schedule
 - ▶ Scale is set to increase/decrease based on the specific time
 - ▶ Scale based on demand
 - ▶ Based on specific parameters (e.g. CPU, memory) you can scale in or scale out your resources based on a scaling policy

Scaling Policies

- ▶ Generally, there are two scaling policies to specify:
 1. Scaling out - launching new instances
 2. Scaling in - terminating instances
- ▶ For example:
 - ▶ Scale out your instances when the CPU utilization goes beyond 70% for a certain amount of time (e.g. 15 minutes)
 - ▶ Terminate instances (Scale in) if CPU utilization drops below 50% after a "cool down" time of 5 minutes
- ▶ Scaling in and out is an example of horizontal scaling



Understanding Horizontal vs. Vertical Scaling

18

▶ Vertical Scaling (Scale up)

- ▶ Increase power of existing server by adding more resources (e.g. CPUs, memory)
- ▶ Pros - Quick and easy fix to add capacity
- ▶ Cons - Costly and introduces single point of failure. Server might reach a limit where it cannot be further upgraded



Scale Up



▶ Horizontal Scaling (Scale out)

- ▶ Increase power of existing system by adding more servers
- ▶ Pros - Less costly, fault tolerant
- ▶ Cons - Requires load balancing, can be more complex to manage



Scale Out

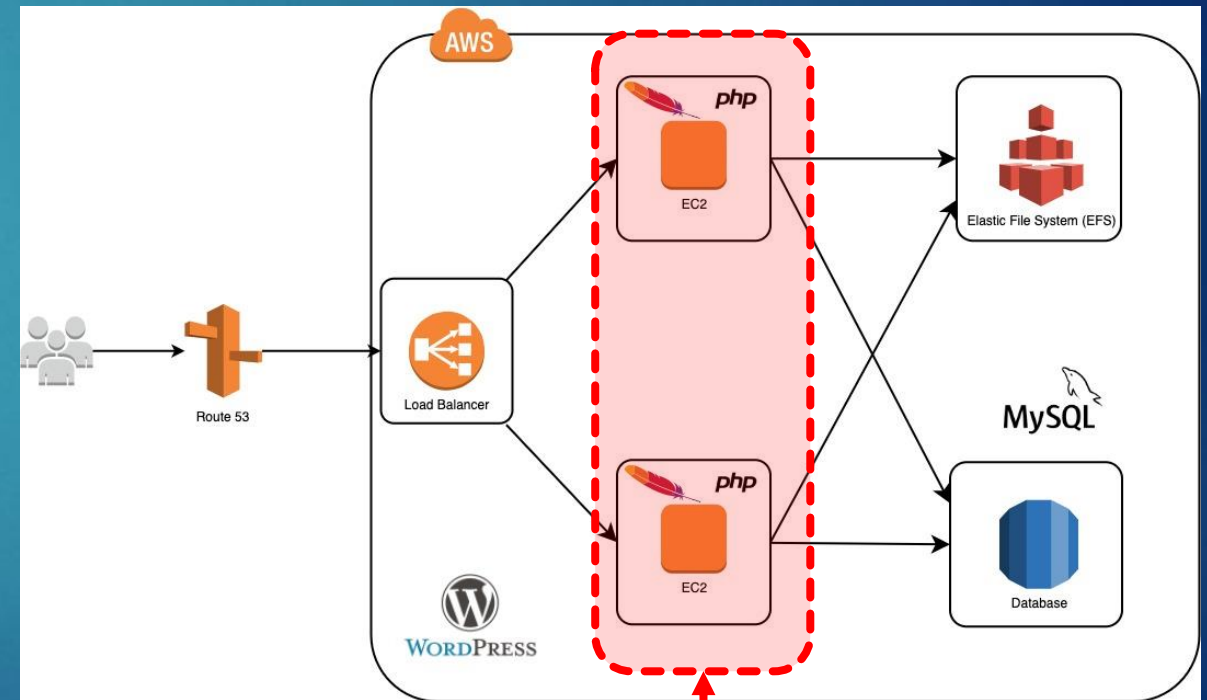
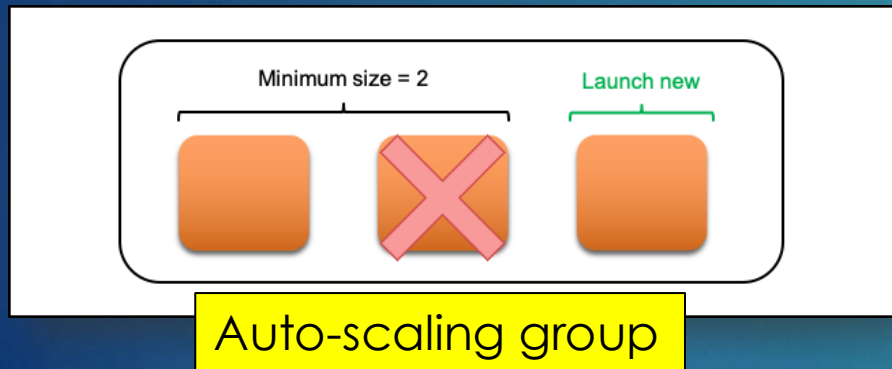
Auto-Scaling Plan – Demo

1. Always maintain current instance level
 - ▶ Configure and maintain a specified number of running instances at all the time. If a server goes down, a new one is launched
2. Scale based on schedule
 - ▶ Scale is set to increase/decrease based on the specific time
3. Scale based on demand
 - ▶ Most advanced scaling model, resources scales by using a scaling policy.
 - ▶ Based on specific parameters (e.g. CPU, memory) you can scale in or scale out your resources.

Auto-Scaling – Demo

20

- This example shows an example of resiliency, which is a very common use for Auto-Scaling

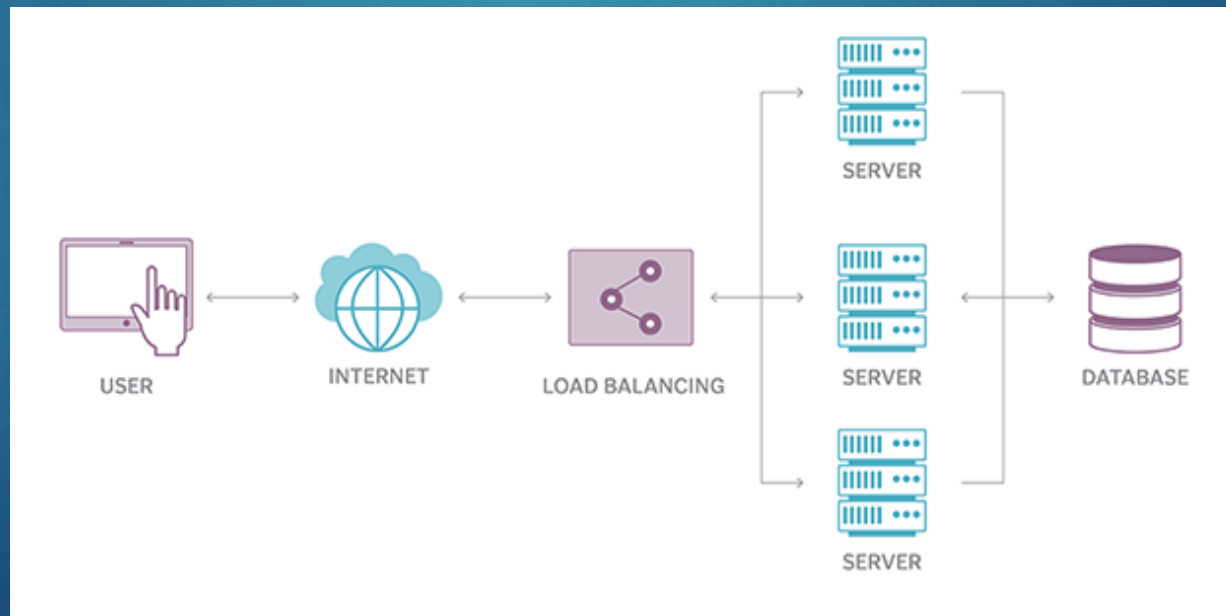


Term Project

- ▶ We will be using Github organizations for the term project
 - ▶ Teams will create a repo in one of their Github accounts and invite me and both CAs:
 - ▶ Follow instructions from the “GitHub Organization Team Configuration” activity
- ▶ Google docs
 - ▶ Each team needs to create a Google shared drive
 - ▶ Invite your team, instructor and CAs
 - ▶ Due by next class

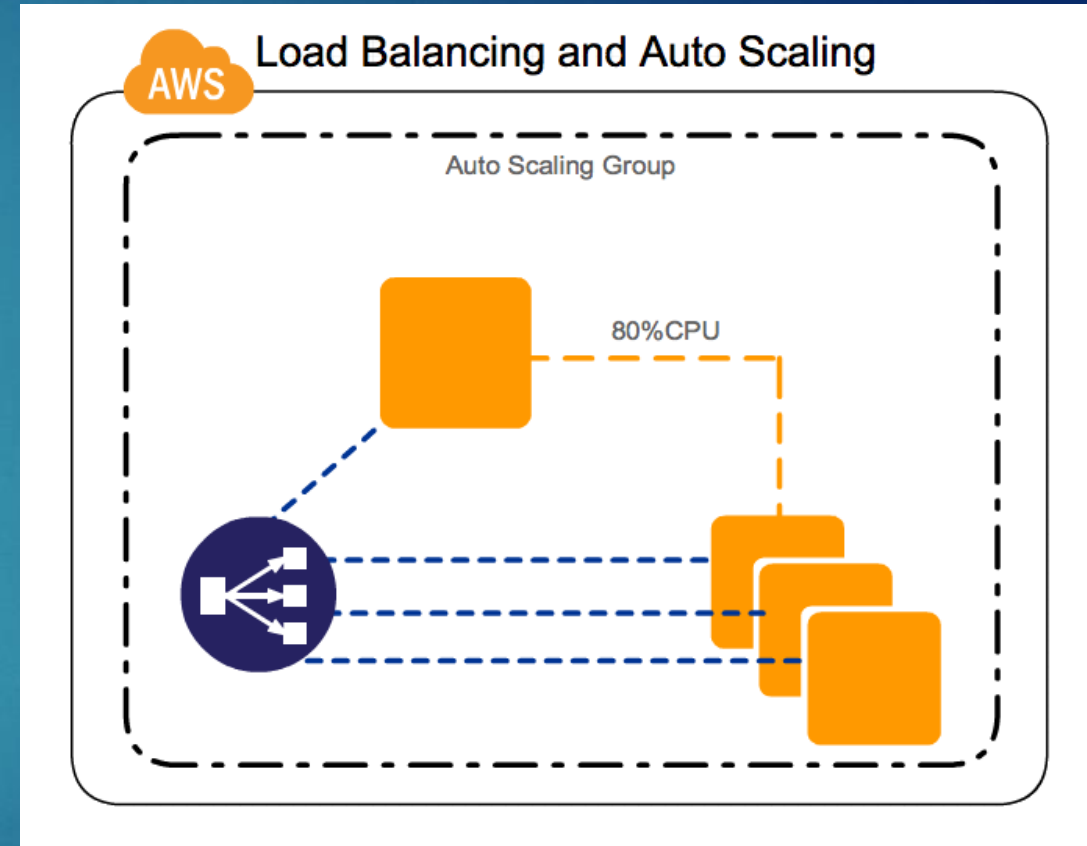
Load Balancing

- ▶ Load balancing involves distributing network traffic across multiple servers
- ▶ This ensures that no single server is overwhelmed with too much demand
- ▶ By evenly distributing the load, load balancing enhances application responsiveness



Load Balancing + Auto-scaling = Better Resiliency

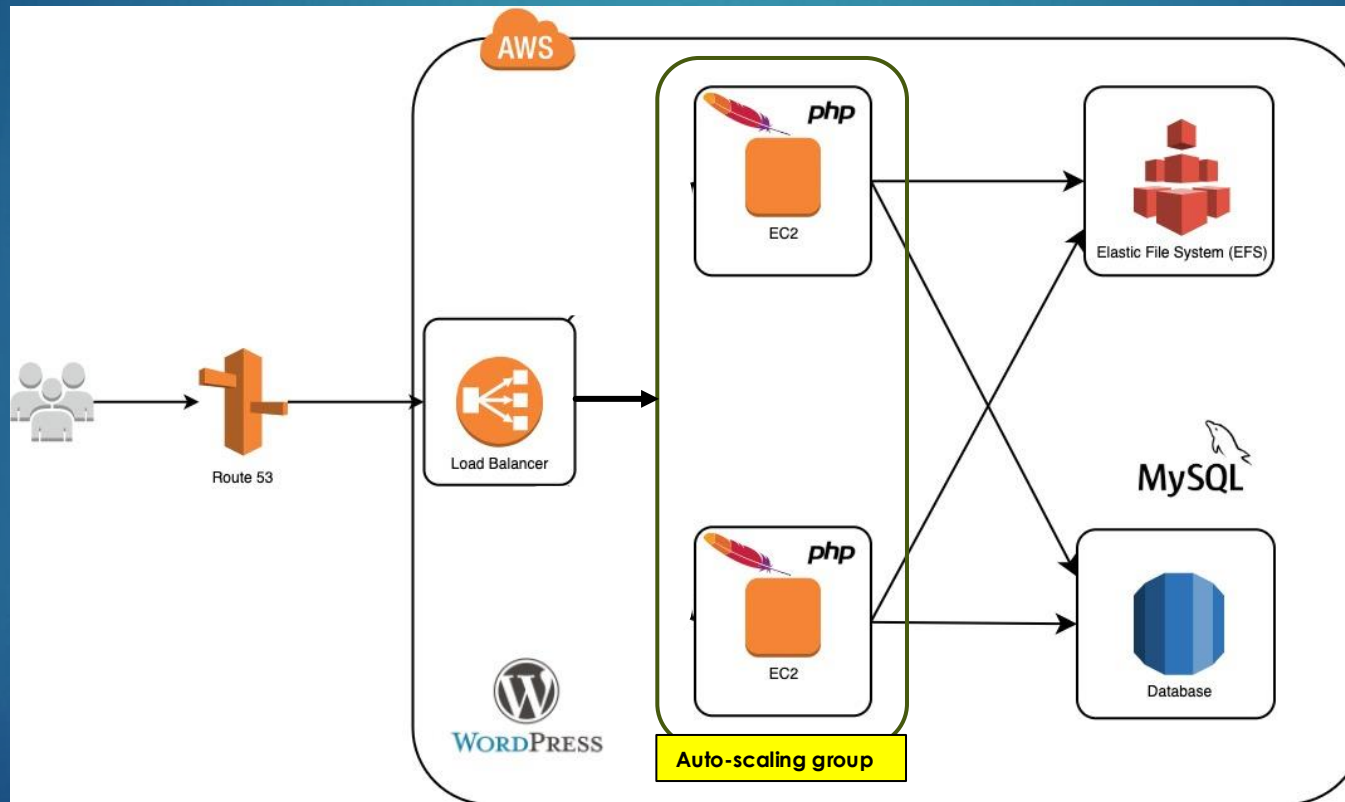
- ▶ With load balancing alone, you need to know ahead of time how much capacity you need so you can keep additional instances running and registered with the load balancer to serve higher loads
- ▶ By associating a load balancer to an Auto-Scaling group, applications can quickly scale-up to handle the increased demand
- ▶ For example, the instance usage goes up to 80% CPU load, the Auto-scaling group in AWS spawns up a new instance and the load balancer attached to the group automatically diverts client traffic to it



Load Balancing + Auto-scaling = Better Resiliency

24

- ▶ When the load goes down, AWS automatically scales the instances down
- ▶ This all happens automatically with no downtime



Auto-Scaling – Challenges

- ▶ There is no one-size-fits all, so it takes time to determine the optimal plan
- ▶ An instance takes time to start up so lowering the threshold of when to add more instances is a good idea
 - ▶ When terminating instances, this should not be done too quickly until it's clear the demand won't quickly increase
- ▶ Measuring a CPU utilization is just one metric and you may consider other factors:
 - ▶ Memory or Network traffic?
 - ▶ Messaging accumulating on queues?
- ▶ AWS offers other tooling that can help to better define when to properly Auto-Scale
 - ▶ CloudWatch is an example, more on this later



Auto-Scaling with Terraform

26

- ▶ Auto-Scaling can be part of your template when building out your cluster

```
# Create an Auto Scaling group
resource "aws_autoscaling_group" "web_asg" {
  name                  = "web-asg"
  max_size              = 2
  min_size              = 2
  desired_capacity      = 2
  health_check_grace_period = 300
  health_check_type     = "ELB"
  vpc_zone_identifier   = ["subnet-abcd1234", "subnet-efgh5678"]
  default_cooldown      = 60 # Cooldown period of 60 seconds
  load_balancer_names   = [aws_elb.web_elb.name]
  launch_template {
    id      = aws_launch_template.web_lt.id
    version = "$Latest"
  }
}
```

Auto-Scale Group

The Auto-Scale plan

Associate with Load Balancer

Launch Template

```
# Create a launch template
resource "aws_launch_template" "web_lt" {
  name_prefix = "web-"
  image_id    = data.aws_ami.amazon_linux_2.image_id
  instance_type = "t2.micro"
}
```

Auto-Scaling & Netflix



- ▶ What type of Auto-Scaling plan and policy does Netflix use?
- ▶ They built their own called “Scrier”
 - ▶ Predicts what the needs will be prior to the time of need and provisions the instances based on those predictions
 - ▶ This differs from AWS Auto-Scaling, which is more reactive
- ▶ Why didn't AWS Auto-Scaling address their needs?
 - ▶ Rapid Spike in demand – Instances take 10-15 minutes to startup, which can leave them vulnerable
 - ▶ Outages - A sudden drop in traffic from an outage is sometime followed by a “retry storm”. A reactive model would cause further problems



Auto-Scaling & Netflix



- ▶ Hybrid Approach – Best of both worlds?
 - ▶ Scryer alone does not replace AWS Auto-Scaling
 - ▶ Netflix has indicated that by combining Scryer + AWS Auto-Scaling together, they are able to handle a wider set of scenarios for more efficient scaling
- ▶ Netflix does open-source many of their solutions they have implemented on AWS
 - ▶ For more information, check out: <https://netflix.github.io/>
 - ▶ Scryer not there (yet)
 - ▶ More examples later





- ▶ From the previous exercise you provided with the AWS Cost Calculator, you've been asked to investigate opportunities to use Auto-Scaling in order to reduce the costs from your initial estimate
- ▶ You've been tasked to identify the following:
 - ▶ What Auto-Scaling usage pattern applies to your scenario?
 - ▶ What type of factors impact this usage pattern?
 - ▶ What type of Auto-Scaling plan would you use to scale in or out?
 - ▶ What would you Auto-Scale on (CPU, memory, other factors)?
- ▶ Download the template from [Assignments > Auto-Scaling Recommendations Template](#)
 - ▶ 1 submission per team
- ▶ Scenarios can be found in [Assignments > Cost Estimating Scenarios](#)



- ▶ Due next class:
 - ▶ For next class, do the Activity in **Assignments > WordPress on Docker**
 - ▶ This is worth 2 points and due at the start of class